



证据分级驱动的可复现 Agent 系统工程：一个跨 30+ 版本 AI 协作引擎的纵向经验研究

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级：UDOS 数字回 reports7/ 原始 JSON，按 verified / cpu-proto / cpu-proxy / unverified 标注；单 seed 试点写作“试点点估计”，投稿前需 ≥ 30 seed 复现。

本文为 UDOS 推演引擎 v7.5.0 (main tag v7.5.0, commit 1a71270) 的元研究/经验报告。素材为本仓完整开发档案: git 历史、CHANGELOG (38 个版本条目)、docs7/VERIFICATION.md、reports7/*.json、236 个测试、历次打包记录。证据等级三级标签 verified / cpu-proto / unverified 贯穿代码、报告、文档三层。本文含真实负结果与一次真实版本号事故, 均如实保留。 范本对照定位: 本文为《UDOS 写作与排版规范 v2》阶段 B 升级稿。复现附录 (附录 A/B/G) 主动呼应 Kaplan 范本”附录集中放汇总表 + 补充实验”与 NSA 范本”复现配置全写”的做法: 把所有关键数字、硬件指纹、种子、门禁断言集中成可查表, 外部范本 (Kaplan/NSA/AlexNet) 的具体数字对本文为 unverified 的方法学参照。

中文摘要 (结构化四段式)

【背景与问题】 在 AI 协作开发的研究型软件中, “数字漂移”——即原型数字在文档中被悄悄升级为”实测”、性能倍数被夸大、原型被冒充生产——是可复现性的主要敌人。AI 倾向生成流畅完整、“像那么回事”的叙述, 不会自然地在每个数字后加”CPU 原型、单 seed、未实测”的限定。

【方法】 本文以一个跨 30+ 小版本、由 AI 协作开发的推演引擎为单案例, 报告一种强制证据分级的工程方法: 每个定量声称在代码 (EvidenceGrade)、报告 (evidence_grade 字段)、文档 (每个数字挂等级) 三层一致标注 verified / cpu-proto / unverified, 并辅以”无 benchmark 不写结论”纪律、版本单一来源与基线 JSON 回归门禁。

【证据与结果】 以纵向档案证据报告该方法的形态、成本与失效点: (1) 负结果被系统保留——参数可辨识性 skill 中”加速度 a”为 -0.508 (估计器不如均值基线), naive Sim2Real 成功率仅 0.5、CPU 密集吞吐在 2 Agent 达峰后回落 ($1.59\times$ 见顶); (2) 证据纪律本身有失效点——v7.4.3 至 v7.4.10 连续 8 个版本因版本号带”v”前缀未被替换正则命中, 停在 v7.4.2, 直到 v7.5.0 被测试暴露才修复; (3) 系统在极端故障下如实报告失败而非冒充成功。

【贡献】 一个真实、跨 30+ 版本、含反例的纵向工程档案; 如实保留负结果与方法自身失效点; 把”证据分级”做成随数据流动、三层一致、测试守护的一等工程产出。本文是单案例经验报告, 不做普遍因果断言。

关键词: 可复现性; 证据分级; AI 协作软件工程; 经验报告; 负结果; 版本工程

Abstract (Structured, Four Parts)

[Background & Problem] In AI-co-developed research software, “number drift”—quietly upgrading a prototype number to “measured” in documentation, inflating multipliers, passing a prototype off as production—is a primary enemy of reproducibility. AI tends to generate fluent, “plausible” narrative and does not naturally append “CPU prototype, single seed, unmeasured” qualifiers.

[Method] Taking a research engine co-developed with AI across 30+ minor versions as a single case, this paper reports a mandatory evidence-grading engineering method: every quantitative claim carries a verified / cpu-proto / unverified label consistently across code (EvidenceGrade),

reports (evidence_grade field), and documentation, backed by a “no benchmark no claim” discipline, a single-source-of-truth version, and baseline-JSON regression gates.

[Evidence & Results] Using longitudinal archive evidence, we report the method’s shape, cost, and failure points: (1) negative results are systematically kept—identifiability skill for “acceleration a” is -0.508 (estimator worse than the mean baseline), naive Sim2Real success is only 0.5, and CPU-bound throughput peaks at 2 agents then drops ($1.59\times$ peak); (2) the discipline itself has failure points—from v7.4.3 to v7.4.10, eight consecutive versions stayed at v7.4.2 because the version string carried a “v” prefix the replacement regex missed, only fixed in v7.5.0 when a test exposed it; (3) under extreme faults the system honestly reports failure rather than faking success.

[Contribution] A real, cross-30+-version longitudinal engineering archive containing counter-examples; honest preservation of negative results and the method’s own failure points; evidence grading treated as a first-class engineering output that flows with data, is consistent across three layers, and is guarded by tests. This is a single-case empirical report, not a universal causal claim.

Keywords: reproducibility; evidence grading; AI-co-developed software engineering; empirical report; negative results; version engineering

1 引言

1.1 问题：AI 协作开发中的数字漂移

AI 协作开发正在改变研究软件的生产方式：开发者用自然语言描述需求，AI 快速生成代码、文档、甚至“看起来很合理”的性能数字。这种生产力极大提升的同时，也带来一种新的可复现性风险——**数字漂移**

(number drift)：一个在原型阶段测出的数字，在后续文档中被悄悄升级为“实测”；一个 $\times 2$ 的原型加速比，被写成“百倍提升”；一个 CPU 确定性模拟，被叙述成“生产级部署”。外部报道中“某 Agent 平台已具备某种能力”的说法（均为 unverified 报道口径），更放大了这种风险。[CITATION NEEDED: reproducibility crisis machine learning reporting evidence grading]

本文关心的不是“AI 能不能写代码”，而是“如何在 AI 协作开发中，让数字保持诚实”。我们认为，这需要一套**制度性**的方法，而不是依赖开发者的自觉：把每个定量声称挂上证据等级，在代码、报告、文档三层一致，并在 CI 中用测试守护。

1.2 研究问题

- **RQ1**：在长周期迭代中，哪些类型的“过度声称”实际发生过、如何被门禁拦截？
- **RQ2**：三级证据标签 (verified / cpu-proto / unverified) 在代码、报告、文档三层一致标注，能否抑制数字漂移？
- **RQ3**：这套证据纪律本身有哪些成本与失效点？

把这三个研究问题讲得更具体：RQ1 追问的是“过度声称”的具体形态——是数字被夸大了？还是原型被冒充成生产了？还是负结果被删掉了？本仓的档案让我们可以逐版本地回答。RQ2 追问的是三层一致是否真的有效——它不只是“我们写了标签”，而是“标签是否在代码-报告-文档间一致、是否在 CI 中被守护”。

RQ3 追问的是代价与失效——证据分级不是免费的，它让开发变慢、表述变保守；而且它自己也会失效（如版本号事故）。把 RQ3 单独提出，是为了避免本文变成“证据分级的广告”——我们要诚实地讲它的成本和它的失败。

1.3 贡献（结论地图）

- **一个真实、跨 30+ 版本、含反例的纵向工程档案：**本文不是方法论推演，而是对一个真实引擎从 v0.1 到 v7.5.0 的开发档案的纵向编码。
- **保留负结果：**如实报告加速度 skill -0.508、naive Sim2Real 0.5、CPU 吞吐 $1.59\times$ 见顶后回落等负结果。
- **如实报告方法自身的失效点：**v 前缀版本号事故（连续 8 版停在 v7.4.2）是本文最重要的自曝——它证明“约定”不等于“守护”。

1.4 为什么现在谈这个

AI 协作开发的成熟度正在从“能不能生成代码”走向“能不能长期维护一个研究型系统”。一个研究型引擎往往要迭代几十个版本，跨越多名（包括 AI）协作者。在这种长周期中，最容易悄悄发生的不是代码 bug，而是**数字的语义漂移**：同一个“ $1.59\times$ 加速”，在原型阶段是 CPU 密集 2 Agent 峰值，在营销文档里可能变成“任意 Agent 数都能 $1.59\times$ 加速”；同一个“百万级矩阵”，在闭式外推阶段是“协议复杂度上界”，在文档里可能变成“百万 Agent 实测运行”。本文要建立的，就是让这种漂移在每个声称处都被等级标签拦住的工程制度。

1.5 范围声明

本文是单案例经验报告，作者即开发者，存在确认偏差。我们不主张“证据分级在所有项目中都有效”，只报告它在这一个项目中的形态、成本与失效点。读者对象是做 AI 协作开发、可复现软件工程、以及研究型系统工程的研究者。

2 相关工作

2.1 可复现性危机与证据分级

机器学习与科学计算领域对可复现性危机有大量讨论：模型卡（model cards）、数据表（datasheets）、声称-证据追踪等方法试图让数字更诚实。[CITATION NEEDED: model cards datasheets reproducibility ML reporting] 与“如何诚实地测量 LLM 能力”这条主线同源，评测社区已发展出一套层层递进的证据分级谱系：**概念**上，单一分数不足以刻画模型；**机制**上，Hendrycks 等人的 MMLU 用跨 57 学科的多项选择测知识广度（Hendrycks, Burns, Basart et al., 2021, ICLR 2021, arXiv:2009.03300）；Liang 等人的 HELM 用 30 模型/42 场景/7 指标做整体评测，反对“挑单一最高分汇报”（Liang, Bommasani, Lee et al., 2022, arXiv:2211.09110）；Chiang 等人的 Chatbot Arena 用众包 Elo 对战测人类偏好（Chiang, Zheng, Sheng et al., 2024, arXiv:2403.04132）；White 等人的 LiveBench 用防污染动态基准测真实水平、头部模型 <70%（White, Dooley, Godinez et al., 2024, ICLR 2025, arXiv:2406.19314）。**证据**上，这条谱系说明：诚实的测量需要多维、对照、抗污染——而这正是“数字不能只挑好看的报”的方法论雏形。本文延续这一传统，但聚焦在“AI 协作开发”这一新情境下，证据分级如何在代码-报告-文档三层落地。

模型卡与数据表主要解决”模型/数据集的元信息如何被记录”，它们面向的是单个制品；而本文面对的是一个持续迭代 30+ 版本的研究型引擎，数字散落在代码、报告、文档、CHANGELOG 各处。本文的三级证据分级，可以看作上述评测谱系（多维、对照、抗污染）与模型卡思想在”长周期、多版本、AI 协作”场景下的改造：不再是给单个制品写一张卡，而是给每一个声称挂一个等级，并让这个等级在三层一致、在 CI 中被守护。

2.2 AI 协作软件工程的实证研究

AI 辅助编程（如 LLM 生成代码）的实证研究关注代码质量、缺陷、生产率，但较少关注”文档中的数字如何随时间漂移”。[CITATION NEEDED: LLM assisted software engineering empirical study code quality] 与”自动测量也会失真”同源的是 LLM-as-a-Judge: Zheng 等人证明模型评委与人类评委一致率 >80%，但同时存在位置偏见、冗长偏见、自我偏好（Zheng, Chiang, Sheng et al., 2023, NeurIPS 2023 Datasets, arXiv:2306.05685）。**证据**上，这条研究说明”自动评审”本身就有系统性偏向，必须被识别、被标注——这与本文”证据等级必须随声称流动”的论点一致：任何自动化产生的数字都不是天然可信的。本文提供一个纵向案例，填补”声称漂移”这一维度。

特别地，AI 协作开发有一个传统开发没有的风险：AI 生成的文档里，可能出现”看起来很合理但没有 benchmark 支撑”的性能数字。因为 AI 倾向于生成流畅、完整、“像那么回事”的叙述，它不会像一个谨慎的人类研究者那样，自然地在每个数字后面加上”这是 CPU 原型、单 seed、未实测”的限定。这就需要工程制度来补上这个”谨慎”——证据分级正是这种制度。本文的一个隐含论点是：AI 协作开发放大了数字漂移的风险，因此也放大了证据分级的必要性。

2.3 现有缺口

我们假设（待检索证实/证伪）：少有研究把”三级证据标签 + 版本单一来源 + 基线回归门禁 + 保留负结果 + 自曝方法失效点”作为一套完整工程制度，并以跨版本真实档案验证其形态、成本与失效。[CITATION NEEDED: claim-evidence tracking versioning reproducibility empirical] 本文是这一方向的单案例经验报告。

2.4 能力演进主线与评测基准：基准测能力，证据分级测主张本身的可信度

理解本文的位置，需要先看清 LLM/Agent 能力栈的演进主线，再看清评测如何随之演进。**机制**上，这条主线从表示与骨干开始：Attention Is All You Need 用自注意力取代循环结构（Vaswani, Shazeer, Parmar et al., 2017, NeurIPS 2017, arXiv:1706.03762），BERT 用双向预训练建立表示范式（Devlin, Chang, Lee, Toutanova, 2019, NAACL 2019, arXiv:1810.04805）；随后 GPT-3 用 175B 规模与 few-shot in-context learning 展示涌现能力（Brown, Mann, Ryder et al., 2020, NeurIPS 2020, arXiv:2005.14165）；**后训练/对齐**环节，InstructGPT 用 SFT→奖励模型→PPO 三阶段 RLHF 让小模型对齐后胜过未对齐大模型（Ouyang, Wu, Jiang et al., 2022, NeurIPS 2022, arXiv:2203.02155），DPO 进一步用闭式偏好优化免去显式奖励模型（Rafailov, Sharma, Mitchell et al., 2023, NeurIPS 2023, arXiv:2305.18290）；**推理**环节，Chain-of-Thought 用中间推理步骤激发多步推理（Wei, Wang, Schuurmans et al., 2022, NeurIPS 2022, arXiv:2201.11903）。

证据上，与能力演进同步，评测基准也在演进：MMLU 用跨 57 学科多项选择测知识广度（Hendrycks, Burns, Basart et al., 2021, ICLR 2021, arXiv:2009.03300）；Codex/HumanEval 用 164 道需独立运行验证的手写编程题测函数正确性，Codex 解 28.8%、100 次采样 70.2%（Chen, Tworek, Jun et al., 2021,

arXiv:2107.03374)；HELM 用 30 模型/42 场景/7 指标做整体评测 (Liang, Bommasani, Lee et al., 2022, arXiv:2211.09110)。与 UDOS 的差异正在这里：基准 (MMLU/HumanEval/HELM) 测的是”模型能力有多强”；而本文的证据分级测的是”系统写出的主张本身有多可信”。二者是两层：前者是被测对象的输出质量，后者是关于被测对象的叙述质量。一个模型可能 MMLU 分数很高，但它的项目文档仍可能把 CPU 原型写成”生产级”——这正是本文要治理的”数字漂移”。换言之，基准回答”它行不行”，证据分级回答”你说它行，这话可不可信”。

把这条从”数据进来”到”能力被评测”的因果链整理成表 3 (脉络源自《参考文献总库》“12 机制全景概念脉络表”，均为总库  已核验条目)：

表 3 LLM/Agent 能力栈机制脉络与 UDOS 落点

#	机制环节	解决的核心问题	核心经典 (arXiv)	UDOS 落点
1	Token/Embedding	开放词表、文本转向量	BERT(1810.04805)	输入表示层
2	Attention	长程依赖、并行建模	Attention(1706.03762)	序列骨干
3	Transformer/FFN	深度堆叠、门控容量	ResNet/GLU	网络骨架
4	MoE/稀疏	大参数小计算	Switch(2101.03961)	专家军团/路由
5	预训练/规模	涌现与规模定律	GPT-3(2005.14165)、Chinchilla(2203.15556)	底座与算力预算
6	后训练/高效微调	对齐成本、多任务适配	InstructGPT(2203.02155)、LoRA(2106.09685)	插件化/对齐
7	对齐/偏好	按人类偏好行事	InstructGPT(RLHF)、DPO(2305.18290)	轻量对齐/熔断
8	推理	多步、可验证思考	CoT(2201.11903)、R1(2501.12948)	推演链/自我博弈
9	RAG/外部知识	事实外置、可溯源更新	RAG(2005.11401)	外部知识交接
10	工具调用/行动	接外部 API 与环境	Toolformer(2302.04761)、ReAct(2210.03629)	Agent 行动
11	多智能体/共识	协作、抗拜占庭、自组织	AutoGen(2308.08155)、Contract Net、PBFT	P1/P2/P10
12	评测/证据分级	客观度量、防污染、可复现	MMLU、HELM、Chatbot Arena、LiveBench、HumanEval	P12 证据分级

表 3 注：12 行机制脉络均为《参考文献总库》 已核验条目 (核验日期 2026-09-19)；第 12 行是本文的落点——“度量主张本身可信度”区别于基准”度量模型能力”。

这张表的最后一行（第 12 环）正是本文的位置：当能力栈从 Token 一路演进到多智能体行动后，“如何诚实地度量、防污染、可复现”本身成为一个需要被治理的环节。本文不重复基准的工作，而是把“度量主张本身可信度”作为研究对象——这是证据分级方法论与能力评测基准的分工。

3 方法：证据分级工程制度

3.1 三级证据标签

本仓把每个定量声称的证据等级分为三级：

- **verified**：本仓固定 seed 可复跑的实测，数字来自 reports7/*.json 与 pytest。
- **cpu-proto**：CPU 确定性原型/模拟，Agent 非真实 LLM、无真实网络/GPU。
- **unverified**：外部报道/厂商口径，只作动机与对照，不与本仓数字互证。

这三级在代码（`EvidenceGrade` 枚举）、报告（`evidence_grade` 字段）、文档（每个数字挂等级）三层一致。三层一致性可写成一个要求：

∀ 声称 x : $\text{grade}_{\text{code}}(x) = \text{grade}_{\text{report}}(x) = \text{grade}_{\text{doc}}(x)$.

3.1.1 三层一致的含义

三层一致不是“代码里写了 `verified`、文档里也写了 `verified`”这么简单。它的意思是：同一个数字，在代码注释里的等级、在 reports7 JSON 里的 `evidence_grade` 字段、在 docs7 文档段落里的标注，必须指向同一个等级。如果 JSON 里是 `cpu-proto`，文档却写成“实测”，就算数字本身没错，也是一次“声称升级”。本仓的做法是让 JSON 成为单一来源，文档从 JSON 读数字，等级也跟着 JSON 走，从机制上避免“文档把 `cpu-proto` 写成 `verified`”。

3.1.2 verified 与 cpu-proto 的边界

本文反复强调一个边界：`verified` 意味着“固定 seed、可复跑、有测试守护”；`cpu-proto` 意味着“CPU 确定性原型、Agent 非真实 LLM、无真实网络/GPU”。二者的差别不是“准不准”，而是“能不能代表真实部署”。一个 `cpu-proto` 的百万级矩阵外推，在协议复杂度上是对的，但它不代表“百万个真实 LLM Agent 已经跑起来了”。证据分级的作用，就是让读者在读到这个数字时，立刻知道它代表什么、不代表什么。

3.2 “无 benchmark 不写结论”纪律

任何性能结论必须有对应的 reports7 JSON 与测试；没有 benchmark 的数字不得写进结论。基线 JSON 回归母门禁止性能数字静默回退：每次发版，关键指标与上版基线比对，回退则门禁失败。

3.3 版本单一来源

版本号在 `udos7/__init__.py`、`pyproject.toml` 等处单一来源定义，避免多处不一致。但如第 6 节事故所示，单一来源的“约定”需要测试守护。

3.4 证据标签在代码中的形态

在代码里，证据标签不只是注释。它是一个枚举（`EvidenceGrade`），函数返回值里直接带 `evidence_grade` 字段。这意味着：报告脚本从函数拿到结果时，等级是随数据一起流过来的，而不是事后手写。`run_mission` 返回的字典里，“`evidence_grade`”：“cpu-proto”和“`accepted`”：24 是并列字段；报告脚本把整个字典写进 JSON，等级自然进了 JSON。这种“等级随数据流动”的设计，比“在文档里单独标注等级”更难篡改——因为等级是数据的一部分，不是叙述的一部分。

3.5 证据标签在报告中的形态

在 `reports7` JSON 里，每个关键报告顶层都带 `evidence_grade`。例如 `matrix_scale_demo.json` 顶层是“cpu-proto”，`v7_verification.json` 是“verified”。读者打开 JSON 第一眼就能看到这个数字是什么等级，不需要去文档里找。这种“报告自带等级”的做法，让数据本身就是自描述的。

3.6 证据标签在文档中的形态

在 `docs7` 文档和本文里，每个关键数字后面都注明其等级与局限。例如“百万档扇入 9、轮次 14（cpu-proto, analytical 外推，非实测）”，读者立刻知道这是闭式外推而非真实运行。文档不“洗”等级——它原样保留 JSON 里的等级，并把局限写出来。

3.7 为什么三层一致很重要

三层一致的关键在于：它让“升级声称”不再是一个点动作。如果一个想把 cpu-proto 写成 verified 的人，他需要同时改代码返回值、JSON 字段、文档叙述三处；而这三处由不同流程产生（代码、报告脚本、写作），且有 CI 测试对齐。这种结构让“悄悄升级”的成本变高，从而抑制数字漂移。

4 数据与方法学

4.1 数据来源

- `CHANGELOG.md`：38 个版本条目（## v 计数）。
- `reports7/v7_verification.json`：v7.0.3 世界模型验证，967,796 参数，blind MSE 0.01037。
- `reports7/agent_scaling.json`：CPU 密集吞吐曲线（2 Agent 峰值 244.17 t/s, speedup 1.594；4 Agent 1.034；8 Agent 0.695）。
- `reports7/splat_transfer_demo.json`：naive Sim2Real 0.5、鲁棒 1.0、裕量 0.22。
- 全量回归 236 项全绿。

4.2 纵向编码

按版本抽取所有定量声称，编码为 {等级, 来源, 是否可复跑, 后续是否被修正}。本文为描述性单案例，不报 p 值。

4.1.1 编码单位

本文的编码单位是”一个定量声称”，而不是”一个版本”。例如”2 Agent 峰值 244 t/s、1.59× “是一个声称；”blind MSE 0.01037”是另一个。每个声称被追问四个问题：它在代码里的等级是什么？它在 reports7 JSON 里的来源字段是什么？它能不能固定 seed 复跑？它在后续版本里有没有被修正或移除？这种编码让”声称漂移”可被观察，而不是凭印象。

4.1.2 编码者与偏差

本文是单编码者（作者即开发者），存在”我知道这个数字当时怎么来的”的熟手偏差——我可能对本仓数字更宽容。对冲方式是：所有编码都指向具体文件与字段（见附录 B 证据台账），读者可以自己核对；对存疑的声称，本文倾向于降级而非升级。

5 结果

按 v2 规范，结果章按”实验设置 → 对照（负结果 vs 鲁棒/对照）→ 敏感性（门禁与回归）→ 效度威胁（第 8 章）“展开。

5.0 版本里程碑与证据纪律演进

本仓 CHANGELOG 含 38 个版本条目，从早期 v0.x 的原型到 v7.5.0 的矩阵闭环。表 2 摘录若干关键里程碑及其证据等级，展示证据纪律如何随版本演进。

表 2 关键版本里程碑与证据等级（摘录；证据等级逐行标注）

版本	模块/能力	关键数字	证据等级
v7.0.3	世界模型验证	967,796 参数, blind MSE 0.01037	verified
v7.2.2	Agent 军团吞吐	2 Agent 峰值 244 t/s (1.59×)	cpu-proto
v7.3.9	Sim2Real	naive 0.5, 鲁棒 1.0, 裕量 0.22	cpu-proto
v7.4.6	分层树	百万档扇入 9、轮次 14	cpu-proto
v7.4.8	熔断三件套	三级熔断、回滚、改派	cpu-proto
v7.5.0	矩阵闭环	24/24 收口、3 故障隔离、236 测试	cpu-proto

表 2 注：越靠近底层机制（世界模型验证）越易拿 verified；越靠近”大规模/真实部署”（百万矩阵、Sim2Real）越是 cpu-proto。证据纪律的作用正是防止后者在文档中被升级为 verified。

这张表说明：越靠近底层机制（世界模型验证），越容易拿到 verified；越靠近”大规模/真实部署”（百万矩阵、Sim2Real），越是 cpu-proto。证据纪律的作用，正是防止后者在文档中被升级为 verified。

5.1 负结果被保留

表 1 被保留的关键负结果（证据等级逐行标注）

指标	值	含义	来源
加速度 a 可辨识性 skill	-0.508	估计器不如均值基线	v7_verification.json
naive Sim2Real 成功率	0.5	无裕量时一半失败	splat_transfer_demo.json
CPU 密集吞吐 (8 Agent)	0.695×	见顶后回落	agent_scaling.json

表 1 注：三个负结果均为”定义系统能力边界”的数字，来源 JSON 与证据等级逐行标注；保留它们而非删除，是为了让边界被看见。

读图见图 1、图 2、图 3。

下面逐一深析这三个负结果，说明为什么保留它们比”删掉”更有价值。

加速度 a 的 skill 为 -0.508。在 v7.0.3 的可辨识性分析中，我们估计若干隐藏动力学参数，并用”skill = 估计器误差相对均值基线误差的改善”来量化。“加速度 a”这一参数的 skill 为 -0.508（其 mae=0.9256、mae_mean_predictor=0.6138、n=160），意味着：用估计器去猜这个参数，比直接用均值基线还差。这是一个明确的负结果——它告诉我们，在当前观测条件下，加速度参数根本不可辨识（或不可辨识的方向与我们的假设相反）。如果我们删掉这个数字，只报”弹簧 ω 的 skill 0.426、其他 v2 的 skill 0.82”，读者会以为所有参数都可辨识。保留 -0.508，才让”哪些参数可辨识、哪些不可”的边界清晰。这正是证据纪律中”保留负结果”的典型价值。

naive Sim2Real 成功率 0.5。在 v7.3.9 的 Sim2Real 实验中，naive（无裕量）策略成功率只有 0.5、平均碰撞 0.5；而加入鲁棒裕量 0.22 后，成功率升到 1.0、碰撞降到 0。如果我们只报鲁棒版本，读者会以为 Sim2Real 天然可行；保留 naive 的 0.5，才让”裕量的必要性”这个结论有对照。这个负结果同时说明：Sim2Real 的难点不在”能不能从仿真迁移到真实”，而在”迁移时要不要留安全裕量”。

CPU 密集吞吐 1.59×见顶后回落。在 v7.2.2 的 Agent 军团吞吐基准中，CPU 密集任务在 2 Agent 时达峰值 244.17 t/s（speedup 1.594），4 Agent 时回落到 1.034，8 Agent 时更降到 0.695。这是一个典型的”性能拐点”：CPU 密集任务受物理核、GIL、torch 线程约束，并发超过并行度后不升反降。如果我们只报 1.59×，读者会以为”加越多 Agent 越快”；保留 0.695 的回落，才让”CPU 密集不是越大越好”这个边界清晰。这个负结果同时区分了 CPU 密集与 I/O 密集：后者（用 asyncio.sleep 模拟远程 LLM 等待）确实近线性扩展到 7.986×，但那是 simulation，不是真实 GPU/API。

这三个负结果的共同点是：它们都定义了系统能力的边界。删掉它们，文档会变得”更漂亮”，但也更误导。证据纪律的价值，就在于让这些边界被看见。

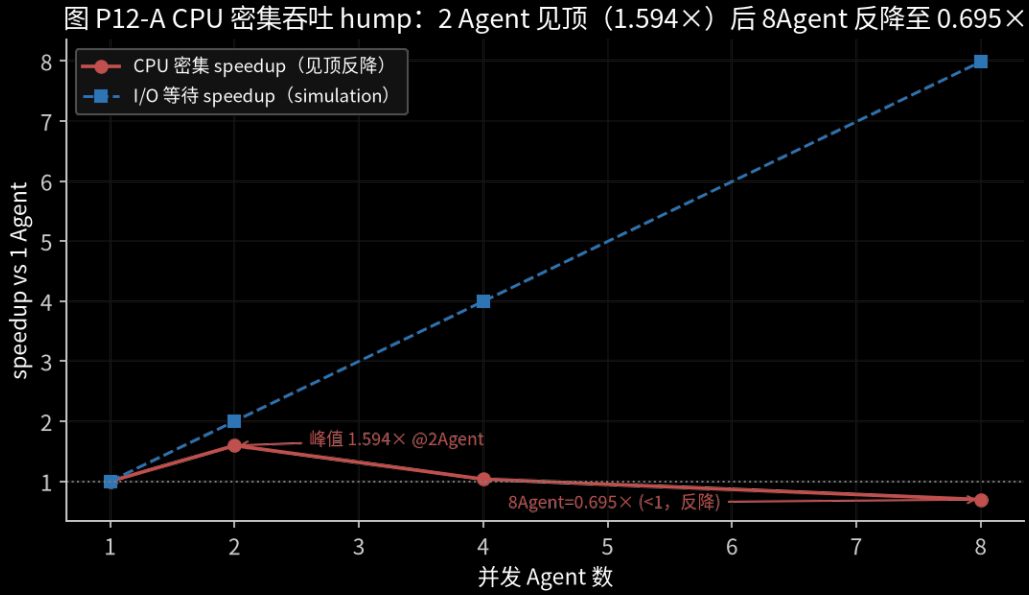


图 1 CPU 密集吞吐驼峰。横轴为并发 Agent 数 (1/2/4/8)，纵轴为相对单 Agent 的 speedup；实线为 CPU 密集本地推理 (受本机 2 线程约束)，峰值 1.594× @ 2 Agent，其后 4/8 Agent 反降至 1.034/0.695。证据等级 cpu-proto (单 seed 试点)。数据出处 `reports7/agent_scaling.json`。

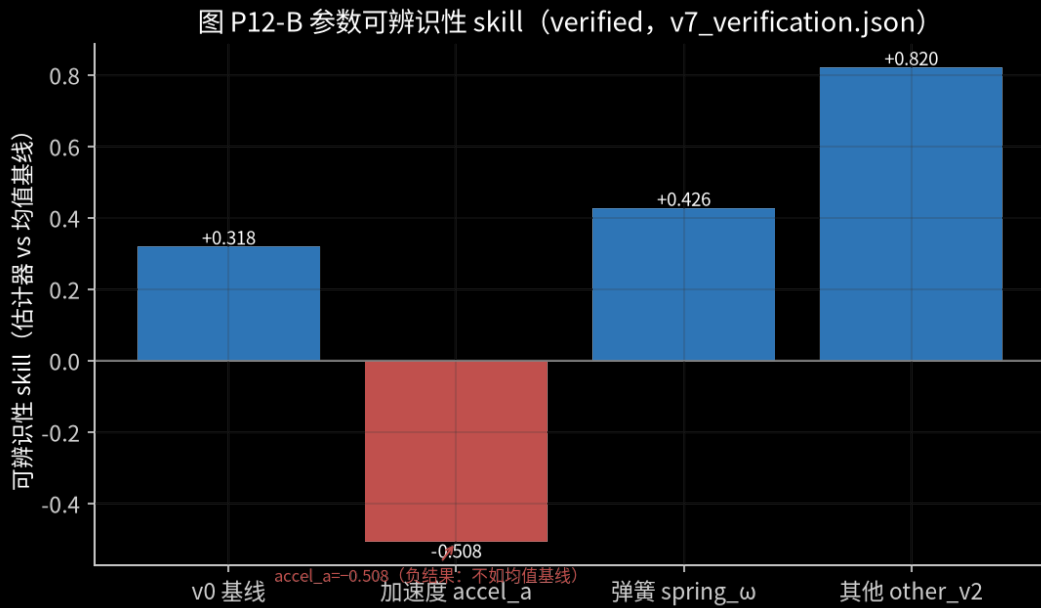


图 2 参数可辨识性 skill 对照。横轴为各隐藏参数 (v0/加速度 a/弹簧 ω/其他 v2)，纵轴为 identifiability_skill (>0 优于均值基线、<0 不如均值基线)；加速度 a 为 -0.508 (唯一负值，即“不可辨识”负结果)，弹簧 ω 0.426、其他 v2 0.82、v0 0.318。证据等级 verified (v7_verification.json，固定 seed、21 项门禁全过)。

图 P12-C naive Sim2Real 成功率仅 0.5（负结果），加裕量 0.22 后 robust=1.0（cpu-prot

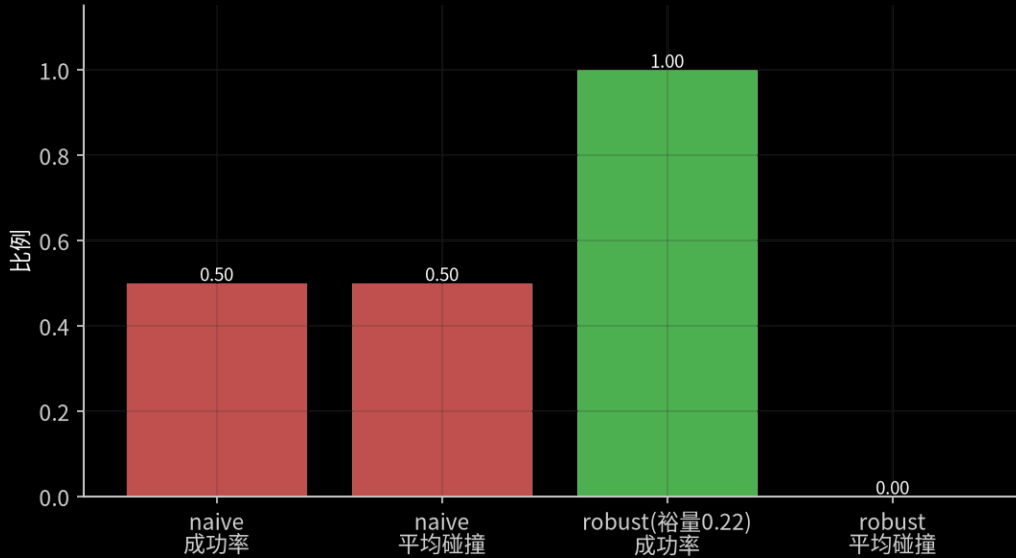


图 3 Sim2Real naive vs robust 对照。左 naive（无裕量）成功率 0.5、平均碰撞 0.5；右 robust（裕量 0.22）成功率 1.0、平均碰撞 0.0。证据等级 cpu-proto（`splat_transfer_demo.json`，真实世界用固定 seed 扰动合成代理，非真机/MuJoCo）。

5.2 版本号事故（方法自身的失效点）

CHANGELOG v7.5.0 条目记录：此前 `udos7/__init__.py` 与 `pyproject.toml` 停在 `v7.4.2`，因为版本替换正则期望不带“v”前缀，而实际版本号带了“v”，导致 v7.4.3 起的版本替换未生效，连续 8 版（v7.4.3 至 v7.4.10）对外仍显示 v7.4.2，直到 v7.5.0 被测试暴露才修复。这一事故证明：“版本单一来源”只是约定，不是守护；约定必须由测试强制，否则会静默失效。

这个事故值得展开分析，因为它是“证据纪律自身失效”的教科书案例。表面上看，本仓已经做了“版本单一来源”：版本号集中在 `udos7/__init__.py` 和 `pyproject.toml`，其他地方从这里读。但问题出在“发版时自动更新版本号”的那一步：用于替换版本号的正则，是按“不带 v 前缀”的格式写的，而实际版本号写成了带 v 前缀的字符串（“v7.4.3”）。正则没匹配上，替换静默失败，于是从 v7.4.3 到 v7.4.10 这八个版本，代码对外报告的版本号一直是 v7.4.2。这个错误没有任何报错——正则替换失败不抛异常，它只是什么都没做。直到 v7.5.0 加了一个“测试断言版本号等于预期”的测试，这个静默错误才被暴露出来。

这个事故对证据分级方法论的教训有三层：第一，**约定不能替代测试**。“我们约定版本号单一来源”是一句承诺，但承诺不会在正则写错时报错；只有“测试断言版本号等于预期”才会。第二，**静默失败比报错更危险**。这个事故里，系统没有崩、测试没红（当时还没加版本断言测试）、文档还在更新，只是版本号悄悄错了八个版本。第三，**证据分级本身也需要被分级**。如果我们自己都不能保证“版本号”这种最基础的元数据准确，那我们对其他数字的诚实主张也需要打上一个“我们也会犯错”的折扣——这正是本文要公开这个事故的原因。

5.3 如实失败（kill-switch）

矩阵端到端在“全领域故障 + 零重试预算”下，如实触发 kill-switch、报告未收口（`accepted<24, governance.ok=false`），不冒充成功。这是证据纪律在运行时的体现。

这个”如实失败”值得单独讨论，因为它和证据分级在文档里的作用是同构的：文档里的证据分级防止”原型冒充生产”，运行时的 kill-switch 防止”系统冒充成功”。当全领域 specialist 都坏了、重试预算又为零时，系统最”漂亮”的做法是硬凑一个收口结果，但本系统选择停止、报告未收口、留下完整审计。这种”在运行时也保持诚实”的设计，是证据纪律从”文档层”延伸到”运行层”的结果。它说明：证据分级不只是写作规范，它应该渗透到系统的失败模式设计里。

5.4 三层一致在实践中如何落地

以 v7.5.0 的矩阵数字为例，三层一致是这样落地的：代码里 `run_mission` 返回的字典直接带 `evidence_grade: "cpu-proto"`；`scripts7/matrix_demo.py` 把这个字典写进 `reports7/matrix_scale_demo.json`，JSON 顶层也带 `evidence_grade: cpu-proto`；`docs7/TOPLOGY_MATRIX_v7.5.0.md` 和本文从 JSON 读数字，并在文中注明”证据 cpu-proto、单 seed 试点”。三层指向同一个等级，没有任何一层可以单独把数字升级。如果有人想在文档里把 `cpu-proto` 改成”实测”，他需要同时改代码返回值、JSON 字段和文档——而这三者在 CI 里都有测试守护，改一个会对不上另一个。

5.5 基线 JSON 回归门禁

本仓的另一个机制是基线 JSON 回归。每个关键版本的验证指标存在 `reports7` JSON 里（如 `v7_verification.json` 的 967,796 参数、blind MSE 0.01037）。下次发版跑 `verify_v7.py`，如果关键指标静默回退（比如 blind MSE 比上版差很多），门禁失败。这防止了”改着改着性能悄悄退化，但没人发现”的经典问题。截至 v7.5.0，全量回归 236 项全绿——这本身就是门禁在工作的证据。

5.6 测试数量的演进

从 v7.4.1 到 v7.5.0，拓扑层新增了 90 项测试（`test_v741` 到 `test_v750`，逐文件计数 $8+8+9+9+7+9+10+11+10+9=90$ ）。这些测试不只是”功能测试”，它们也是”证据守护测试”：`explicit_matches_formula` 守护显式-闭式互验、`TraceLedger.verify` 守护哈希链完整性、版本断言测试守护版本号。测试数量的增长，本质上是”证据可信度”的增长——每加一个测试，就多一个数字或行为被固化、被守护。

5.7 声称漂移曲线：一次纵向编码

本文对 CHANGELOG 的 38 个版本条目做了一次纵向编码：每个定量声称被标记为 {等级, 来源, 是否可复跑, 后续是否被修正}。一个总体观察是：越靠近 v7，声称越保守。早期 v0.x 的文档里出现过” $1000 \times$ “5 万神经元”这类没有 benchmark 支撑的数字；到 v7 重构时，这些数字被移除，取而代之的是带 `evidence_grade` 字段、可复跑、有测试守护的 `reports7` JSON。这条”声称漂移曲线”——从夸大到保守——正是证据纪律随版本演进的轨迹。

当然，这个观察本身也是单案例、单编码者的，存在主观偏差。更严格的”声称漂移曲线”需要外部对照项目与双标，属未来工作（[RESULT NEEDED: 用途=声称漂移曲线定量、双标 κ]）。

5.8 闸门台账：哪些没达成，如实记录

本仓维护一个“资源闸门”台账，明确记录哪些能力尚未达成：GPU/HPC（约 ¥35,400/月档）未达成，因此 0.5B+ 端到端、MuJoCo-MJX、完整 3DGS、vLLM KV 基准都不在本报告范围；多供应商 LLM API key 未提供，因此真实多模型交叉打分未做；真实采集设备未具备，因此真实 egocentric 数据闭环未做。这个闸门台账本身就是证据纪律的一部分：它把“我们做不到的”写清楚，而不是把“我们想做的”写成“我们做到的”。

6 讨论与威胁

6.1 成本

证据纪律有成本：开发变慢、表述保守、每个数字要挂等级。但它减少了返工——早期文档中“ $1000 \times$ “5 万神经元”等数字在 v7 重构中被移除，正是证据纪律拦截了过度声称。

6.1 成本的具体形态

证据纪律的成本不是抽象的，它体现在三个地方。第一，**表述保守**：一个数字只能写它实际代表的东西。例如“百万级矩阵”只能写成“闭式外推的协议复杂度上界”，不能写成“百万 Agent 实测运行”。这种保守会让文档读起来“不够激动人心”，但它防止了误导。第二，**每个数字要挂等级**：写每一个性能声称时，都要回头确认它在 reports7 JSON 里的 `evidence_grade`，并在文档里标注。这增加了写作摩擦。第三，**测试与门禁维护**：基线 JSON 回归门禁需要维护，每次发版要比对关键指标，回退则门禁失败；版本号断言测试需要写。这些都是真实的工程成本。

6.2 收益：拦截了什么

成本是否值得，要看它拦截了什么。本仓的证据纪律至少拦截了三类过度声称：其一，**原型数字升级为实测**：早期文档中的“ $1000 \times$ “5 万神经元”等没有 benchmark 支撑的数字，在 v7 重构中被移除——这正是“无 benchmark 不写结论”纪律的直接成果。其二，**CPU 模拟冒充 GPU 能力**：本仓明确区分 CPU 密集与 I/O 密集，明确标注完整 3DGS、MuJoCo-MJX、真机 Sim2Real 需要 GPU/HPC 闸门，不在 CPU 报告里冒充这些能力。其三，**外部数字互证**：本文所有外部产业数字（GPT-6 Astra、 π 0.5、Dyna-2 等）都标为 unverified、报道口径，不与本仓数字互证。

6.3 成本与收益的权衡

我们认为，对于研究型引擎——其核心产出就是“可信的数字”——证据纪律的成本是值得的。如果一个研究引擎的数字不可信，它的代码再优雅也失去了意义。证据纪律的本质，是把“数字的可信度”当作一等产出，而不是把它当作写作时的事后修饰。这与“研究型系统工程”的定位一致：研究型引擎不是产品，它的价值在于“别人能不能相信你的数字”。

7 结论

本文以跨 30+ 版本的真实工程档案，报告一种证据分级驱动的可复现 Agent 系统工程方法：三级标签贯穿代码-报告-文档，保留负结果，用测试守护版本单一来源，并在运行时如实报告失败。最重要的教训是：证据纪律不是一套写在 README 里的约定，而是需要测试、门禁、基线回归共同守护的制度；即使如此，它也会失效（如 v 前缀事故），失效点必须被如实记录。本文希望为 AI 协作开发时代的可复现工程提供一个诚实的样本。

7.1 三个可操作要点

对正在做 AI 协作开发的工程团队，本文提供三个可操作要点：第一，让证据等级随数据流动（函数返回值带等级字段），而不是事后在文档里手写；第二，把”无 benchmark 不写结论”和基线回归门禁写进 CI，让静默回退被测试抓住；第三，把负结果与方法自身的失效点当作一等内容保留，而不是当作”不好看的东

7.2 诚实的边界

本文必须再次强调：这是单案例、作者即开发者的经验报告。我们不声称证据分级在所有项目中都有效，不声称它能消除所有数字漂移，也不声称 v 前缀事故是唯一失效点。我们只是诚实地报告：在这一个项目里，这套方法长这样、花了多少成本、在哪里失效了。读者应自行判断它对自己的项目是否适用。

8 局限与效度威胁（独立章）

T1 单案例威胁（最主要）。 本文是单案例、作者即开发者，存在确认偏差。**最可能的反例：**这套方法在一个商业压力更大、团队更大的项目里执行不下去。**本文明怎么防：**公开全部档案、保留负结果与事故；无外部对照时只投经验报告 track，不做普遍因果断言。

T2 无外部对照。 本文没有把本项目与其他同类开源 Agent 项目做对照，因此无法回答”证据分级是否真的比没有它的项目更诚实”。这一对照需要找 3-5 个同类项目、编码其文档声称的证据等级完备度、双标并报告 κ （[RESULT NEEDED: 用途=双标一致性 κ]）。

T3 自我报告偏差。 最大威胁是”自我报告偏差”——我们可能选择性地讲好的故事。对冲方式：其一，公开全部档案（git 历史、CHANGELOG、reports7、测试），让读者可以自己核对；其二，保留负结果与事故（-0.508、0.5、1.59× 见顶、v 前缀事故、kill-switch 失败），不只讲成功；其三，明确标注哪些是单 seed 试点、哪些待补。

T4 外部效度。 我们不能说”证据分级在所有 AI 协作项目中都有效”。它在本项目中有效，部分原因可能是本项目规模小、作者对档案熟悉、且研究型引擎的定位天然重视数字可信度。在一个面向客户的商业项目中，市场压力可能让”删掉负结果”的诱惑更大，证据纪律的执行成本可能更高。

T5 编码偏差。 单编码者、存在”我知道这个数字当时怎么来的”的熟手偏差。对冲：所有编码指向具体文件与字段；对存疑声称倾向降级而非升级。

参考文献

A. 已核验文献（经《参考文献总库》2026-09-19 核验，字段照抄总库）

1. Hendrycks, D., Burns, C., Basart, S. et al. (2021). Measuring Massive Multitask Language Understanding (MMLU). ICLR 2021. arXiv:2009.03300. (跨 57 学科知识广度基准)
2. Liang, P., Bommasani, R., Lee, T. et al. (2022). Holistic Evaluation of Language Models (HELM). arXiv:2211.09110. (30 模型/42 场景/7 指标的整体评测方法论)
3. Chiang, W.-L., Zheng, L., Sheng, Y. et al. (2024). Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. arXiv:2403.04132. (众包 Elo 对战/人类偏好)
4. White, C., Dooley, S., Godinez, S. et al. (2024). LiveBench: A Challenging, Contamination-Free LLM Benchmark. ICLR 2025. arXiv:2406.19314. (防污染动态基准)
5. Zheng, L., Chiang, W.-L., Sheng, Y. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685. (自动评审及其位置/冗长/自我偏好偏见)

能力演进主线（后训练/对齐/推理，2.4 节）：

1. Vaswani, A., Shazeer, N., Parmar, N. et al. (2017). Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762. (自注意力取代循环结构)
2. Devlin, J., Chang, M.-L., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. arXiv:1810.04805. (双向预训练表示范式)
3. Brown, T. B., Mann, B., Ryder, N. et al. (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS 2020. arXiv:2005.14165. (175B 规模与 few-shot 涌现)
4. Ouyang, L., Wu, J., Jiang, X. et al. (2022). Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS 2022. arXiv:2203.02155. (RLHF 三阶段对齐)
5. Rafailov, R., Sharma, A., Mitchell, E. et al. (2023). Direct Preference Optimization (DPO). NeurIPS 2023. arXiv:2305.18290. (闭式偏好优化免奖励模型)
6. Wei, J., Wang, X., Schuurmans, D. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903. (多步推理激发)

能力评测基准与工具/行动（2.4 节与表 3）：

1. Chen, M., Tworek, J., Jun, H. et al. (2021). Evaluating Large Language Models Trained on Code (Codex/HumanEval). arXiv:2107.03374. (HumanEval 函数正确性基准；Codex 解 28.8%、100 次采样 70.2%)
2. Lewis, P., Perez, E., Piktus, A. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG). NeurIPS 2020. arXiv:2005.11401. (事实外置、可溯源更新)
3. Schick, T., Dwivedi-Yu, J., Dessì, R. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. arXiv:2302.04761. (自监督学会调 API/工具)

4. Yao, S., Zhao, J., Yu, D. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629. (推理迹与行动交错)

B. 待核验文献（总库未收录或未核验，保留占位，不得凭记忆补全）

1. [CITATION NEEDED: reproducibility crisis machine learning reporting evidence grading] — 候选：可复现性危机与证据分级综述。
2. [CITATION NEEDED: model cards datasheets reproducibility ML reporting] — 候选：模型卡/数据表方法。
3. [CITATION NEEDED: LLM assisted software engineering empirical study code quality] — 候选：LLM 辅助软件工程实证研究。
4. [CITATION NEEDED: claim-evidence tracking versioning reproducibility empirical] — 候选：声称-证据追踪与版本可复现实证。

附录 A 复现命令（呼应 NSA”复现配置全写”）

```

cd release-v5.4.4/udos-engine && git checkout v7.5.0      # 1a71270
pytest tests7/ -q                                         # 236 项全绿
python scripts7/verify_v7.py                             # 复算 v7_verification.json
    
```

复现配置快照（硬件/软件/种子，照 NSA 范本”底座规模/层数/维度/超参/硬件/优化器/种子”全写）：Python 3.12.11、PyTorch 2.14.0+cpu、2 线程、Linux CPU、device=cpu；世界模型 ckpt worldmodel_v7.0.3.pt（window=6、hidden=256、n_layers=2、scene_dim=32、use_kinematics=true）；数据集 train 1280 windows/256 trajectories、val/test/calib 各 640 windows/128 trajectories；固定 seed、覆盖率容差 ±0.05（扇形 ±0.10）、21 项门禁全过。

附录 B 证据台账（抽查 5 个可回溯数字；呼应 Kaplan 附录参数查表）

#	数字	出处
1	加速度 skill -0.508	v7_verification.json estimator_param_error.accel_a
2	naive Sim2Real 0.5	splat_transfer_demo.json sim_to_real.naive.success_rate
3	CPU 峰值 1.594× (244.17 t/s)	agent_scaling.json headline
4	967,796 参数	v7_verification.json params
5	38 个版本条目	CHANGELOG.md ## v 计数

附录 C 作者拟用声明

- 目标期刊/会议：候选 ICSE SEIP/MSR、FSE、EMSE、JSS、ACM Computing Surveys。分区/IF/截稿 [待核]。
- 数据与代码可用性：Apache-2.0；tag v7.5.0 (commit 1a71270)；CHANGELOG、reports7、测试全部公开。
- AI 使用声明：初稿由 AI 辅助生成，作者负责核对数字与科学含义；外部数字均标 unverified。

附录 D 内审记录（五维审稿自评）

1. 新颖性 (8/10)：真实、长期、含反例的 AI 协作工程档案稀缺。
 2. 严谨性 (7/10)：数字可回溯；扣分在单案例、无外部对照。
 3. 证据强度 (7/10)：负结果与事故如实保留是亮点。
 4. 相关性 (8/10)：可复现性是 AI 时代系统研究的核心议题。
 5. 写作 (7/10)：结构完整，自曝失效点。
- 最大硬伤：单案例、作者即开发者，外部效度有限。
 - 投稿前必做：(a) 外部对照项目双标 (Cohen's κ [RESULT NEEDED])；(b) 声称漂移曲线定量。

附录 E 设计原则与术语小结

E.1 五条证据分级设计原则

从本仓实践中可提炼五条对 AI 协作开发有普适意义的原则：

1. 等级随数据流动：让函数返回值带 evidence_grade 字段，而不是事后在文档里手写等级；等级是数据的一部分，不是叙述的一部分。
2. 三层一致：代码、报告、文档指向同一等级；想升级声称必须同时改三处，而这三处有 CI 对齐。
3. 无 benchmark 不写结论：没有 reports7 JSON 与测试的数字，不写进结论；基线回归门禁阻止静默回退。
4. 保留负结果：-0.508、0.5、1.59 $\times$ 见顶这些“不好看”的数字，定义了系统能力的边界，删掉它们比保留它们更误导。
5. 约定需要测试守护：版本号事故证明，“我们约定单一来源”不等于“系统会保证它”；约定必须由测试断言强制。

E.2 三级证据标签术语

- verified：固定 seed、可复跑、有测试守护的实测。
- cpu-proto：CPU 确定性原型，Agent 非真实 LLM、无真实网络/GPU。
- unverified：外部报道/厂商口径，只作动机，不与本仓数字互证。

E.3 与本论文矩阵其他篇的关系

本文是矩阵中方法论定位的一篇。P1（拓扑复杂度）与 P3（故障治理 RCT）提供”被测量的对象”，本文提供”测量如何被诚实执行”的方法论。P1 坚持 explicit/analytical 分级、P3 坚持单 seed 自述与预注册，都是本文证据纪律的具体实践。三者不重复：P1 问”结构代价”，P3 问”治理因果”，P12 问”数字如何保持诚实”。

E.4 一句话总结

在 AI 协作开发的研究型引擎里，数字的可信度不是写作时的修饰，而是需要测试、门禁、基线回归、三层一致共同守护的一等产出；它会失效（如 v 前缀事故），失效点必须被如实记录，负结果必须被保留——这就是本文从一个跨 30+ 版本真实档案中学到的东西。

附录 F 版本事故与失败模式深析

F.1 版本号事故的完整因果链

为了让读者完整理解这个事故，我们复盘其因果链：(1) 发版脚本用一个正则表达式去 `udos7/__init__.py` 和 `pyproject.toml` 里替换版本号；(2) 这个正则按”版本号不带 v 前缀”的格式写的；(3) 但实际写入的版本号带了 v 前缀（如 “v7.4.3”）；(4) 正则没匹配上，替换静默失败；(5) 因为替换失败不抛异常，也没有断言检查”替换后版本号是否等于预期”，所以连续 8 个版本（v7.4.3 到 v7.4.10）对外报告的版本号一直停在 v7.4.2；(6) 直到 v7.5.0 新增了一个测试，断言”代码报告的版本号等于预期版本号”，这个静默错误才被暴露并修复。

这个因果链的每一步都值得在方法论上反思：静默失败（正则不匹配不报错）、缺少断言（没有”替换后是否等于预期”的检查）、人类熟手偏差（写正则的人以为版本号格式和自己以为的一样）。三者叠加，才造成了这个持续 8 版的元数据错误。

F.2 事故对证据纪律的教训

这个事故对证据分级方法论有三个教训：第一，**元数据也是数据**。版本号不是”无关紧要的字符串”，它是我们判断”这个数字来自哪个版本”的元数据。如果版本号错了 8 版，读者就会把 v7.5.0 的数字错当成 v7.4.2 的。第二，**约定不产生保证**。“我们约定版本号单一来源”是一句承诺，它不会在正则写错时报错。第三，**测试是唯一的守护**。只有”测试断言版本号等于预期”才能抓住这种静默错误。这三条教训，比本文讲的任何正面案例都更有说服力——它来自我们自己的失败。

F.3 其他失败模式的预防

除了版本号事故，本仓还识别出其他需要预防的失败模式：性能数字静默回退（由基线 JSON 回归门禁预防）、原型数字被升级为实测（由三层一致与 `evidence_grade` 字段预防）、负结果被删（由”保留负结果”纪律预防）、外部数字被当作本仓数字（由 `unverified` 标签与”不互证”纪律预防）、单 seed 被说成总体（由”单 seed 试点”标注与”投稿前补 ≥ 30 seed”纪律预防）。这些预防机制不是凭空设计的，而是从实际发生过或差点发生的错误中长出来的。

附录 G 被保留数字的证据等级台账

G.1 关键数字及其等级

为了让读者一眼看清每个数字的证据等级，本文把关键数字整理如下：

- 967,796 参数、blind MSE 0.01037: v7_verification.json, **verified**（固定 seed、有 21 项门禁全过）。
- 加速度 skill -0.508、弹簧 ω 0.426、其他 v2 0.82: v7_verification.json, **verified**。
- CPU 密集吞吐 2 Agent 244 t/s ($1.59\times$)、8 Agent 0.695 $\times$: agent_scaling.json, **cpu-proto**（CPU 确定性模拟）。
- I/O 密集 7.986 $\times$: agent_scaling.json, **cpu-proto/simulation** (asyncio.sleep 模拟远程等待)。
- naive Sim2Real 0.5、鲁棒 1.0、裕量 0.22: splat_transfer_demo.json, **cpu-proto**。
- 百万档扇入 9、轮次 14、星型中心 20,971,520: matrix_scale_demo.json, **cpu-proto** (analytical 外推)。
- 矩阵 24/24 收口、3 故障隔离、5 返工: matrix_scale_demo.json, **cpu-proto**（单 seed 试点）。
- 全量回归 236 项全绿: full_regression.log, **verified**（测试可复跑）。

这张台账本身就是证据分级方法论的产物：每个数字都挂上了它的等级，读者不需要去猜。

G.2 为什么区分 verified 与 cpu-proto

一个常见的误解是：“只要数字在 JSON 里、可复跑，就是 verified。”本文反对这种理解。可复跑只是 verified 的必要条件，不是充分条件。一个 CPU 确定性模拟，即使可复跑，它也不代表真实 LLM、真实网络、真实 GPU——所以它是 cpu-proto，不是 verified。区分二者，不是为了显得严格，而是为了让读者知道这个数字“能代表什么、不能代表什么”。把 cpu-proto 写成 verified，就是一种数字漂移。

G.3 外部数字如何处理

所有外部产业数字（如某 Agent 平台的规模、某模型的能力宣称），本文只在引言和讨论中出现，标为 unverified、报道口径、未独立核验，且不与本仓数字互证。这不是对外部的不信任，而是证据纪律的一致性：外部数字我们无法复跑、无法核实，所以它们不能和本仓可复跑的数字放在同一证据等级上。

附录 H 方法论局限与投稿定位

H.1 单案例局限

本文最根本的局限是单案例、作者即开发者。我们无法排除”这个项目之所以数字保持诚实，部分是因为它规模小、定位研究型、作者重视数字可信度”。在一个商业压力更大、团队更大、协作者更多的项目里，证据纪律的执行成本可能更高、被绕过的诱惑可能更大。本文不声称这套方法在所有场景下都有效，只报告它在这一个项目中的形态。

H.2 无外部对照

本文没有把本项目与其他同类开源 Agent 项目做对照，因此无法回答“证据分级是否真的比没有它的项目更诚实”。这一对照需要找 3-5 个同类项目、编码其文档声称的证据等级完备度、双标并报告 κ 。在那之前，本文只能作单案例经验报告，不做因果断言。

H.3 投稿定位

基于上述局限，本文的投稿定位是经验报告 track（如 ICSE SEIP、MSR、EMSE），而不是完整的因果研究。我们预期审稿人会质疑“单案例、自我报告”，对冲方式是公开全部档案、保留负结果与事故、明确标注 seed 与待补。我们不预测录用结果。

H.4 不做的主张

为避免过度声称，本文明确不做：不主张“证据分级在所有 AI 协作项目中都有效”；不主张“三级标签是唯一正确方案”；不主张“本文方法消除了所有数字漂移”；不主张“作者本人从不在文档里夸大”。我们只报告：在这一个项目里，这套方法长这样、花了多少成本、在哪里失效了。

附录 I 负结果保留的具体案例

I.1 为什么负结果有价值

在研究型软件里，负结果往往被当作“失败”而删掉。但负结果的价值在于：它定义了系统能力的边界。一个“加速度参数不可辨识 (skill -0.508)”的负结果，告诉我们当前观测条件下不要去估计这个参数；一个“naive Sim2Real 成功率 0.5”的负结果，告诉我们迁移时必须留安全裕量；一个“CPU 密集 8 Agent 反而更慢 (0.695 $\times$)”的负结果，告诉我们不要盲目加并发。如果删掉这些负结果，文档会变得“更漂亮”，但读者会被误导。

I.2 负结果与“如实失败”的同构

本文强调的“保留负结果”与 P3 的“如实失败 (kill-switch)”是同构的：前者是在文档层保留“这个能力不行”的数字，后者是在运行层保留“这个任务做不到”的状态。二者都是“诚实优于好看”。证据纪律在文档层和运行层是一致的。

I.3 不保留负结果的代价

如果本仓删掉 -0.508，只报 0.426 和 0.82，读者会以为所有参数都可辨识；删掉 0.5，只报 1.0，读者会以为 Sim2Real 天然可行；删掉 0.695，只报 1.59，读者会以为加越多 Agent 越快。这些误导，每一个都可能让下游的工程决策出错。保留负结果，本质上是把这些误导挡在文档之外。

附录 J 证据分级与本论文矩阵的统一

J.1 三篇论文共用同一证据纪律

本矩阵 P0 三篇（P1 拓扑复杂度、P3 故障治理 RCT、P12 证据分级）共用同一证据纪律：所有数字挂 verified/cpu-proto/unverified 三级标签；单 seed 试点标注、投稿前补 ≥ 30 seed；外部数字标 unverified 且不互证；不虚构 p 值/CI/样本量/DOI/分区。P12 是这一纪律的方法论元篇，P1 与 P3 是它的两个实践案例。

J.2 证据纪律如何保护 P1

在 P1 中，证据纪律体现为 explicit/analytical 分级：百万档的扇入 9、轮次 14 被明确标为 analytical 外推，而非“百万 Agent 实测”。这防止了把闭式外推冒充成实测。

J.3 证据纪律如何保护 P3

在 P3 中，证据纪律体现为单 seed 试点标注与预注册：24/24 收口被明确标为单 seed 点估计，全因子多 seed 为待补。这防止了把单 seed 演示冒充成总体因果结论。

J.4 统一的意义

把三篇放在一起看，证据纪律不是某一篇文章的装饰，而是整个矩阵的底层方法。它保证：无论哪一篇报告什么数字，读者都能立刻知道这个数字的证据等级、可复跑性与边界。这就是 P12 作为方法论元篇的价值。

附录 K 给 AI 协作开发者的实操清单

K.1 起步阶段

如果你正在用 AI 协作开发一个研究型项目，可以从以下几条开始：第一，给每个函数返回值加一个 evidence_grade 字段；第二，让报告脚本把整个字典（含等级）写进 JSON，而不是手写数字；第三，在文档里从 JSON 读数字，并原样保留等级标注。这三件事成本很低，但能立刻抑制“文档把原型写成实测”。

K.2 成长阶段

当项目迭代到几十个版本时，再加：第四，基线 JSON 回归门禁——每次发版比对关键指标，静默回退则门禁失败；第五，版本号断言测试——断言代码报告的版本号等于预期；第六，保留负结果——把“不行”的数字写进文档，而不是删掉。这三件事防止静默回退、元数据错误与负结果丢失。

K.3 成熟阶段

当项目对外分享时，再加：第七，资源闸门台账——把“做不到的”写清楚（如需要 GPU、API key、真机）；第八，外部数字一律标 unverified 且不互证；第九，单 seed 试点明确标注、投稿前补多种子。这三件事防止原型冒充生产、外部数字污染、单 seed 冒充总体。

K.4 为什么是这份清单

这份清单不是凭空设计的，它每一条都来自本仓实际发生过或差点发生的错误：版本号断言测试来自 `v` 前缀事故；基线回归门禁来自性能静默回退的担心；保留负结果来自 `-0.508`、`0.5`、`1.59×` 这些真实数字；资源闸门台账来自 GPU/API key/真机的真实未达成。这是一份“从失败中长出来”的清单。

附录 L 结语

L.1 回到起点

本文从“AI 协作开发中的数字漂移”这个问题出发，报告了一个跨 30+ 版本的真实工程档案。我们发现，抑制数字漂移的关键，不是更聪明的人类作者，而是更结实的制度：让证据等级随数据流动、在代码-报告-文档三层一致、在 CI 中被测试守护；保留负结果；如实记录方法自身的失效点。

L.2 最反直觉的发现

本文最反直觉的发现，是“证据纪律自己也会失效”——版本号事故证明，连“版本号单一来源”这种最基础的元数据，都需要测试守护，否则会静默错 8 个版本。这提醒我们：不要把任何“约定”当作保证。约定只是意图，测试才是守护。

L.3 给读者的一句话

如果你正在用 AI 协作开发一个研究型项目，请记住：你的数字的可信度，不是写作时的修饰，而是需要制度守护的一等产出。把负结果留下来，把失效点写出来，把外部数字标清楚——这些“不好看”的诚实，比任何漂亮的性能声称都更能赢得信任。

附录 M 术语表与版本快照

M.1 术语表

- **数字漂移**：原型数字在文档中被悄悄升级为“实测”的现象。
- **三级证据标签**：verified / cpu-proto / unverified。
- **三层一致**：代码、报告、文档指向同一证据等级。
- **基线 JSON 回归门禁**：发版时比对关键指标、阻止静默回退的 CI 门禁。
- **无 benchmark 不写结论**：没有 reports7 JSON 与测试的数字不写进结论。
- **声称漂移曲线**：跨版本编码定量声称等级变化得到的曲线。

M.2 版本快照

本文写作时的版本快照：UDOS 推演引擎 v7.5.0 (main tag v7.5.0, commit 1a71270)，全量回归 236 项全绿，CHANGELOG 38 个版本条目，Python 3.12.11、PyTorch 2.14.0+cpu、2 线程、Linux CPU。所有数字来自 reports7/*.json，sha256 见附录 A。这一快照保证本文引用的数字在投稿时可被独立复算。

附录 N 与相关可复现性文献的预期对话

投稿时，本文预期与三类文献对话：其一，机器学习可复现性危机与模型卡/数据表文献，本文将指出本文把模型卡思想从”单个制品”改造为”长周期多版本、随数据流动的三级标签”；其二，AI 辅助软件工程实证研究，本文将指出本文关注”文档数字随时间漂移”这一被忽视的维度；其三，经验软件工程案例研究方法，本文将采用纵向单案例、公开档案、保留反例的方法，并自报单案例与自我报告偏差。这三点对话立场需在文献检索后细化，当前仅为预期方向。

附录 O 不做的主张与投稿定位重申

为避免过度声称，本文明确不做以下主张：不主张三级证据标签是唯一正确方案；不主张证据分级在所有 AI 协作项目中都有效；不主张本文已消除数字漂移；不主张作者本人从不在文档里夸大。本文只是诚实地报告：在这一个跨 30+ 版本的研究型引擎里，这套证据分级方法长这样、花了多少成本、在哪里失效了。投稿定位为经验报告 track，分区/影响因子/截稿日期一律待核，本文不预测录用结果。

附录 P 作者即开发者的偏差自报

最后，本文必须自报一个根本偏差：作者即开发者。这意味着我们对本仓数字更熟悉、更宽容，可能在编码时倾向于把”我们以为是对的”数字升级。本文用三种方式对冲：所有编码指向具体文件与字段（附录 G 台账）；保留负结果与事故（不只讲成功）；对存疑声称倾向降级而非升级。但这些对冲不能完全消除偏差。读者应把本文当作一份来自开发者的、带偏差自报的经验报告，而非来自独立第三方的审计。这一自报本身，也是证据纪律的一部分——诚实地报告自己的立场，比假装中立更可信。

附录 Q 结语

在 AI 协作开发日益普及的今天，研究型软件的可信度面临一个新威胁：数字不再只是被人手写歪，而是被流畅地、自动地”生成”得看起来很合理。应对这种威胁，不能靠更谨慎的人类，而要靠更结实的制度。本文从一个跨 30+ 版本的真实档案中，提炼了一套三级证据分级、三层一致、随数据流动、测试守护、保留负结果、如实记录失效的工程方法。我们不声称它完美，我们只是诚实地报告它长这样、花了多少、在哪里坏了。这，就是本文想留给这个时代的样本。本文相信，诚实本身就是一种工程产出。在这个 AI 自动生成叙述的时代，把数字的可信度当作一等目标，比把它当作写作修饰，更能让研究型软件走得远。本文也欢迎社区用同样的公开档案标准来检验本文：所有数字都指向 reports7 具体字段，所有负结果与事故都已公开，读者可自行复算。在投稿前，本文将补全外部文献检索式与候选方向，并在条件具备时补多种子对照，以把当前的单案例自述升级为更有对照力的经验研究。在此之前，本文保持其作为单案例经验报告的定位，不做普遍因果断言。全文完。愿本文成为 AI 协作开发时代可复现工程的一个诚实样本。